Computer Algorithms

Unit-1

Fundamentals of Algorithms

History of Algorithm

- The word algorithm comes from the name of a Persian author ,Abu Ja'far Mohammed bin Musa al Khowarazimi(825 A.D.).
- This word has taken on a special significance in computer science,
- "algorithm" has come to refer to a method that can be used by a computer for the solution of a problem.
- That is what makes algorithm different from words such as process, technique, or method.

Definition of Algorithm

- An algorithm is a finite set of instructions that, if followed ,accomplishes a particular task.
- All algorithms must satisfy following criteria.
  1. **Input** – Zero or More
  2. **Output** – At lease one
  3. **Definiteness** – Each instruction clear and unambiguous
  4. **Finiteness** – Terminate after finite number of steps
  5. **Effectiveness** – Every instruction must be basic and feasible

Areas of study

1. How to device algorithms

   Divide and Conquer, Greedy Method, Dynamic Programming

2. How to validate algorithms

   Show that it computes correct answer for all possible legal inputs

3. How to analyze algorithms

   Time and space requirement

4. How to test a program

   Debugging and profiling

Debugging

- Process of executing programs on sample data sets
- to determine whether faulty results occur, and if so correct them
- Debugging can only point to the presence of errors, but not to their absence

Profiling (Performance Measurement)

- Process of executing a correct program on data sets
- and measuring the time and space it takes to compute the results
- It helps for improvement

- We can describe an algorithm in many ways .
- We can use natural language like English,

  if we choose this operation , the resulting

  operations instructions are definite.
- Graphical representations called **Flow Charts** are another possibility,

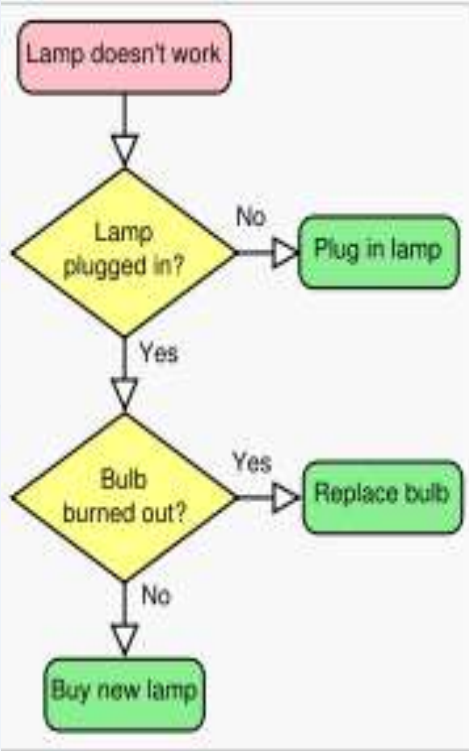
  but they work well only if the algorithm is small and simple.

Difference between Pseudo Code and Flowchart

Algorithm Lamp:

```
Begin
  if  Lamp_plugged_in then
      if Bulb_burned_out then
          Replace bulb
      else buy new lamp.
   else plug in lamp.
   End
```

Pseudo Code



Flow Chart

Algorithm Specification

- Pseudo code Conventions

while ⟨*condition*⟩ **do**
{
⟨*statement* 1⟩
⋮
⟨*statement n*⟩
}

for *variable* := *value*1 **to** *value*2 **step** *step* **do**
{
⟨*statement* 1⟩
⋮
⟨*statement n*⟩
}

A **repeat-until** statement is constructed as follows:

repeat
⟨*statement* 1⟩
⋮
⟨*statement n*⟩
until ⟨*condition*⟩

if ⟨*condition*⟩ **then** ⟨*statement*⟩
if ⟨*condition*⟩ **then** ⟨*statement* 1⟩ **else** ⟨*statement* 2⟩

Algorithm Specification

- Pseudo code Conventions

```
1  Algorithm Max(A, n)
2  // A is an array of size n.
3  {
4      Result := A[1];
5      for i := 2 to n do
6          if A[i] > Result then Result := A[i];
7      return Result;
8  }
```

Recursive Algorithms

- Fibonacci Series
- Factorial of a Number
- GCD
- Towers of Hanoi

Recursive Vs Iterative

RECURSION	ITERATIONS
Recursive function – is a function that is partially defined by itself	Iterative Instructions –are loop based repetitions of a process
Recursion is usually slower than iteration due to overhead of maintaining stack	Iteration does not use stack so it's faster than recursion
Recursion uses more memory than iteration	Iteration consume less memory
Infinite recursion can crash the system	infinite looping uses CPU cycles repeatedly
Recursion makes code smaller	Iteration makes code longer

Recursive Algorithms

- Fibonacci Series

```
int fibonacci(int n)
{
   if (n <= 1)
        return n;
   else
        return (fibonacci(n-1) + fibonacci(n-2));
}
```

Recursive Algorithms

- Factorial of a Number

```
int factorial(int n)
{
   if (n == 0)
        return 1;
   else
        return (n * factorial(n-1));
}
```

Recursive Algorithms

- GCD - **Euclid's Algorithm**

```
int gcd(int m, int n)
{
if ((m % n) == 0)
  return n;
else return gcd(n, m % n);
}
```

gcd(468, 24) gcd(24, 12) => 12

gcd(135, 19) gcd(19, 2) gcd(2, 1) => 1

Recursive Algorithms

- GCD - **Dijkstra's Algorithm**

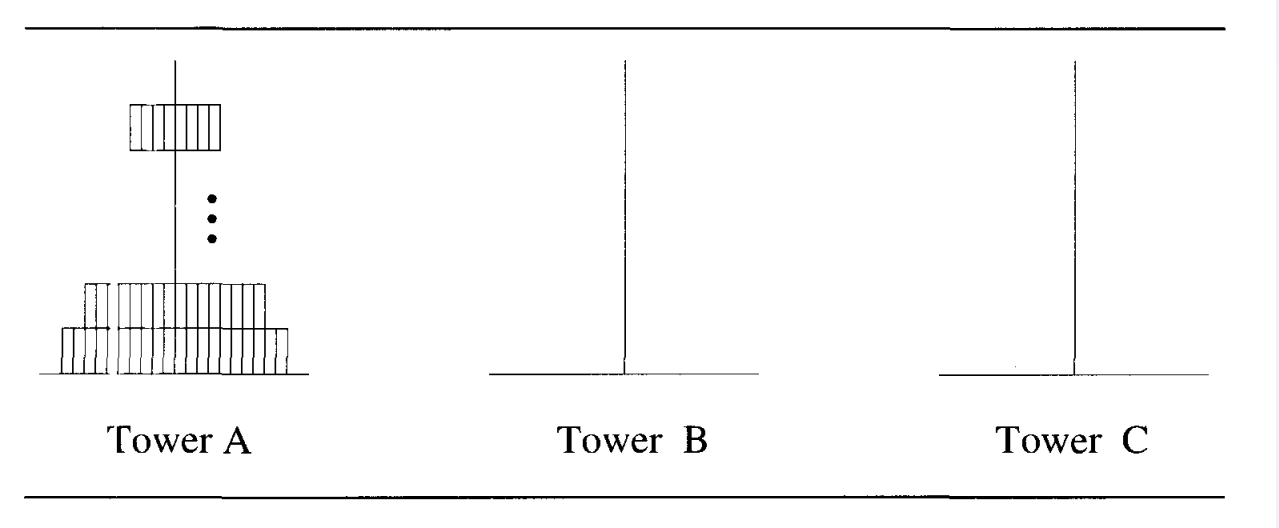
```
int gcd(int m, int n)
{
   if(m == n)
      return m;
   else if (m > n)
      return gcd(m-n, n);
   else
      return gcd(m, n-m);
}
```

Recursive Algorithms

- GCD - **Dijkstra's Algorithm**
- gcd(468, 24)
- gcd(444, 24)
- gcd(420, 24) ...
- gcd(36, 24)
- gcd(12, 24) (Now n is bigger)
- gcd(12, 12) (Same) => 12

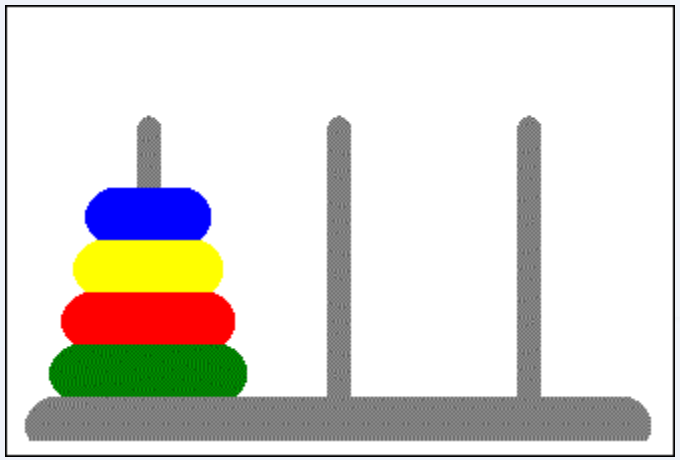
Recursive Algorithms

- Towers of Hanoi



Recursive Algorithms

- Towers of Hanoi



Recursive Algorithms

- Towers of Hanoi

```
procedure Hanoi(n: integer; source, dest, by: char);
Begin
   if (n=1)
     then print('Move the plate from ', source, ' to ', dest
   else
     begin
       Hanoi(n-1, source, by, dest);
       print('Move the plate from ', source, ' to ', dest);
       Hanoi(n-1, by, dest, source);
     end;
End;
```

Recursive Algorithms

```
#include <stdio.h>
void towers(int, char, char, char);
int main()
{ int num;
  printf("Enter the number of disks : ");
  scanf("%d", &num);
  printf("The sequence of moves involved in the
     Tower of Hanoi are :\n");
  towers(num, 'A', 'C', 'B');
  return 0;
}
```

Recursive Algorithms

```
void towers(int num, char frompeg, char topeg,
     char auxpeg)
{ if (num == 1)
  { printf("\n Move disk 1 from peg %c to peg %c",
     frompeg, topeg);
     return;
  }
    towers(num - 1, frompeg, auxpeg, topeg);
    printf("\n Move disk %d from peg %c to peg
     %c", num, frompeg, topeg);
    towers(num - 1, auxpeg, topeg, frompeg);
}
```

Recursive Algorithms

Enter the number of disks : 3
The sequence of moves involved in the Tower of Hanoi are :
Move disk 1 from peg A to peg C
Move disk 2 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 3 from peg A to peg C
Move disk 1 from peg B to peg A
Move disk 2 from peg B to peg C
Move disk 1 from peg A to peg C

Performance Analysis

- Many Criteria upon which we can judge algorithm –
  - Does it do, what we want it to do?
  - Does it work correctly according to original specification of the task
  - Is there documentation that describes how to use it and how it works?
  - Modularity?
  - Is code reachable?

Space / Time Complexity

- The Space Complexity of an algorithm is the amount of memory it needs to run to completion

- The Time Complexity of an algorithm is the amount of computer time it needs to run to completion

- Performance Analysis can be divided into two phases

1. Priori Estimates
   - Performance Analysis

2. Posteriori Testing
   - Performance Measurement

Space Complexity

- Space needed by algorithm is sum of

  1. A fixed part – which is independent of characteristics of the inputs and outputs
  e.g. Instruction Space, Space for variables

  2. A variable part – which consists of space needed by component variables whose size is dependent on the particular problem instance being solved
  e.g. Space needed by reference variable, recursion stack space

  $S(P)=c + Sp$

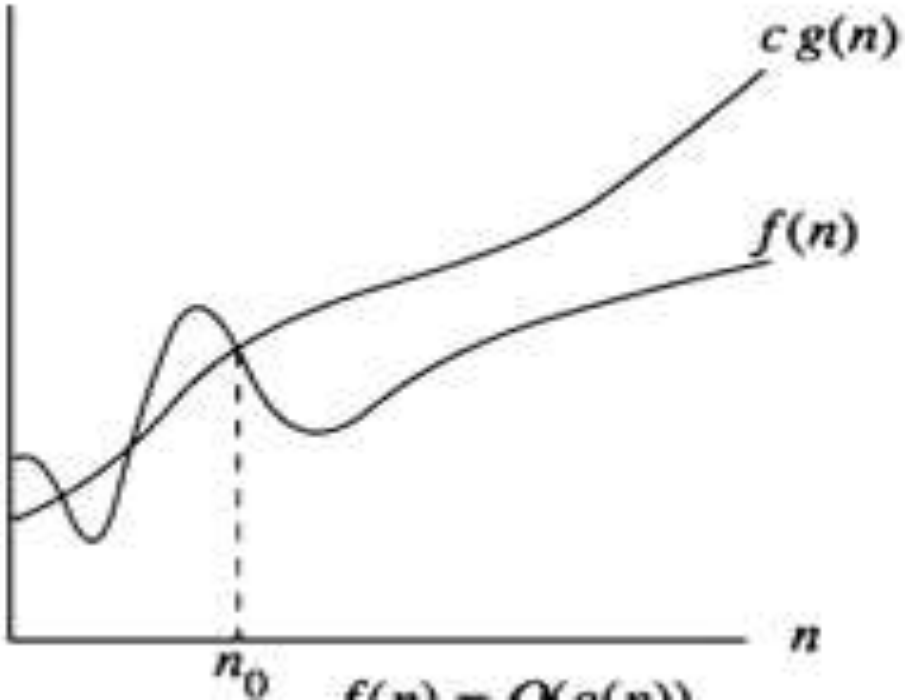
Time Complexity

- The time T(P) taken by a program P is the sum of the compile time and the run time (execution time)
- Compile time does not depend on instance characteristics
- Program compiled Once can be run several times
- So only run time is concerned with Time Complexity t_p
- Key Operations like Comparison are considered

Asymptotic Notation (O, Ω, Θ)

- O (Big oh) –

  The function f(n)= O(g(n))
  iff there exist positive constants c and n_0 such that f(n)<= c * g(n) for all n, n>n_0

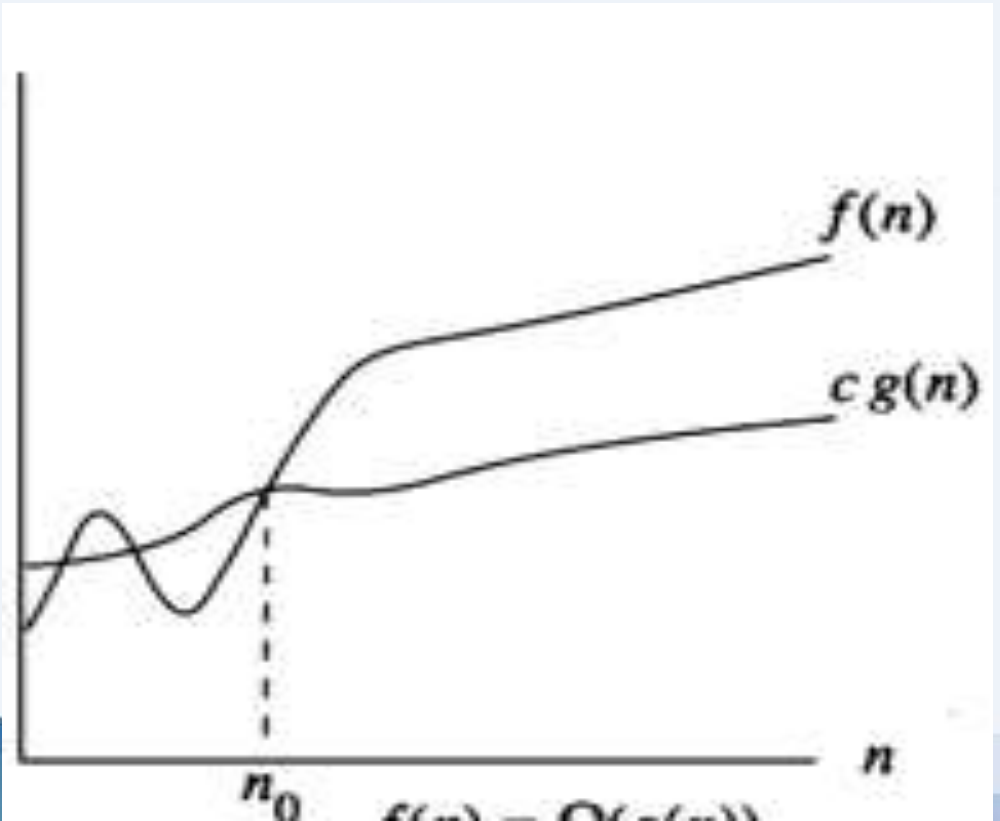


It specifies Upper Bound Worst Case complexity.

Asymptotic Notation (O, Ω, Θ)

- Ω (Omega) –

  The function f(n)= Ω(g(n))
  iff there exist positive constants c and n_0 such that f(n)>= c * g(n) for all n, n>n_0

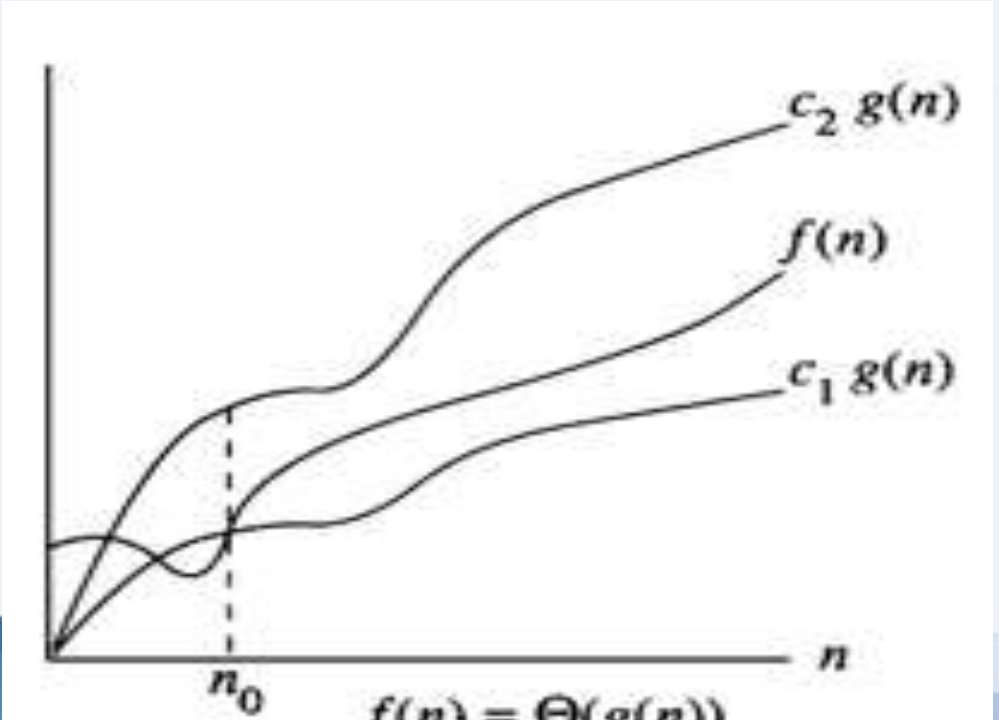


It specifies Lower Bound Best Case complexity.

Asymptotic Notation (O, Ω, Θ)

- Θ (Theta) –

  The function f(n)= Θ(g(n))
  iff there exist positive constants c_1, c_2 and n_0 such that $c_1 * g(n) <= f(n) <= c_2 * g(n)$ for all n, $n > n_0$



It specifies average case complexity.

Difference between best case and worst case complexity is very less.

- o (Little Oh)
  - The function f(n)=o(g(n)) iff
  - $\lim_{n \to \infty} f(n)/g(n) = 0$
- $g(x)$ grows much faster than $f(x)$
- the growth of $f(x)$ is nothing compared to that of $g(x)$.

- ω (Little Omega)
  - The function f(n)= ω(g(n)) iff
  - $\lim_{n \to \infty} g(n)/f(n) = 0$

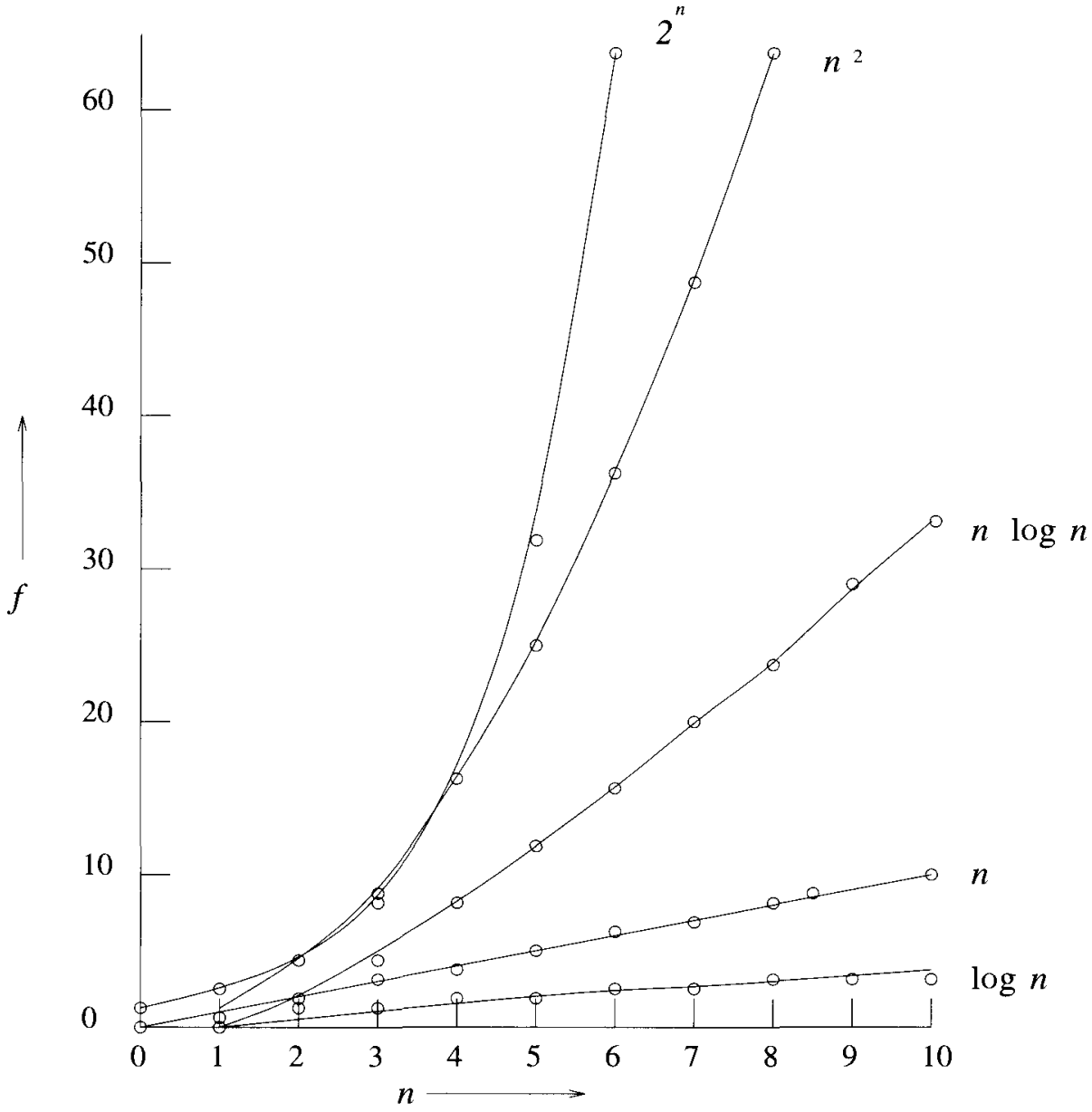
Practical Complexities

Time complexity		Example
O(1)	*constant*	Adding to the front of a linked list
O(log *N*)	*log*	Finding an entry in a sorted array
O(*N*)	*linear*	Finding an entry in an unsorted array
O(*N* log *N*)	*n-log-n*	Sorting n items by ‘divide-and-conquer’
O(N^2)	*quadratic*	Shortest path between two nodes in a graph
O(N^3)	*cubic*	Simultaneous linear equations
O(2^N)	*exponential*	The Towers of Hanoi problem

Practical Complexities

Efficiency	Big-O	Iterations	Estimated Time
Logarithmic	$O(\log n)$	14	microseconds
Linear	$O(n)$	10,000	seconds
Linear logarithmic	$O(n(\log n))$	140,000	seconds
Quadratic	$O(n^2)$	$10{,}000^2$	minutes
Polynomial	$O(n^k)$	$10{,}000^k$	hours
Exponential	$O(c^n)$	$2^{10{,}000}$	intractable
Factorial	$O(n!)$	10,000!	intractable

Measures of Efficiency for $n = 10{,}000$


2^n
n^2
$n \log n$
n
$\log n$
f
n
0
10
20
30
40
50
60
0 1 2 3 4 5 6 7 8 9 10

Function values

log n	n	n log n	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65536
5	32	160	1024	32768	4294967296

RANDOMIZED ALGORITHMS

- Probability Theory
- The probability of an event E is denoted to be
- |E| / |S|, where S is the sample space.

- Mutual exclusion
- Two events E1 and E2 are said to be mutually exclusive
- if they do not have any common sample points,
- that is $E1 \cap E2 = \Phi$

RANDOMIZED ALGORITHMS

- A randomized algorithm is one that makes use of a randomizer
- such as a random number generator.
- Some of the decisions made in the algorithm depend on the output of the randomizer.

RANDOMIZED ALGORITHMS

- Since the output of any randomizer might differ in an unpredictable way from run to run,
- The output of a randomized algorithm could also differ from run to run for the same input.
- The execution time of a randomized algorithm could also vary from run to run for the same input.

RANDOMIZED ALGORITHMS

- Randomized algorithms can be categorized into two classes:
- Las Vegas algorithms
- always produce the same (correct) output for the same input.
- The execution time of a Las Vegas algorithm depends on the output of the randomizer.

RANDOMIZED ALGORITHMS

- Monte Carlo algorithms.
- outputs might differ from run to run for the same Input
- Consider any problem for which there are only two possible answers, say, yes and no.
- If a Monte Carlo algorithm is employed to solve such a problem,
- then the algorithm might give incorrect answers depending on the output of the randomizer.
- We require that the probability of an incorrect answer from a Monte Carlo algorithm be low.

RANDOMIZED ALGORITHMS

- Typically, for a fixed input, a Monte Carlo algorithm does not display much variation in execution time between runs,
- whereas in the case of a Las Vegas algorithm this variation is significant.

Las Vegas algorithms - Example

- Randomized Quick Sort
- Identifying the Repeated Element
- Consider an array a[ ] of n numbers that has n/2 distinct elements and n/2 copies of another element.
- The problem is to identify the repeated element.
- Solutions –
  - n/2+1 unique elements so time is n/2+2
  - Sorting – O(n log n)

Las Vegas algorithms - Example

- Randomized Algorithm
- Generate 2 indices between 1 and n randomly
- Compare the values indicated by these indices
- If values are same, repeated element is found
- Complexity – O(log n)

Monte Carlo algorithms - Example

- Primality Testing
- Given a number n, find if it is prime or not
- Prime number is divisible by 1 and that number
- Test if it is divisible by any number from 1 to root of that number
- If not divisible – it is prime number
- Time required - root of (n)
- Randomized algorithm can be used for primality testing – O(log n)

PRIMALITY TESTING

- Deterministic algorithm for testing whether an integer n is a prime is simply a test whether any of the integers between 2 and root of n divides n.
- If number of digit is more than 40 then complexity is exponential to the input size
- It will take millions of years.
- Recently, M. Agrawal, N. Kayal, and N. Saxena have designed a polynomial algorithm for primality.

PRIMALITY TESTING

- Fermat's little theorem : Let n be a prime number and let a : any number that is not divisible by n; then,

- Based on the primality test given,

$$a^{n-1} \equiv 1 \pmod{n}.$$

PRIMALITY TESTING

- choose a base a at random from {2, ... , n-1}
- and return true (that is, the number is prime) if and only if $a^{n-1} = 1 \pmod{n}$.
- Most composite numbers n fail the Fermat test for many integers a between 2 and n-1.
- Thus, for such numbers the Monte Carlo algorithm has a high probability of being correct.